

Scott Domains in Lean / Mathlib

ScottLean4 · for Dana Scott

Does Lean’s Mathlib formalize Scott domains? The honest picture, from the **v4.32.2** tree: the *foundations* are present (and carry Dana Scott’s name in the source), but the *signature constructions* of domain theory as a model of computation are not.

What Mathlib does have (the foundations)

- **Scott continuity** — `Order/ScottContinuity.lean` (with `Prod` and `Complete` companions): a function preserving directed suprema. The file’s docstring reads “Scott continuous (referring to Dana Scott).”
- **The Scott topology** — `Topology/Order/ScottTopology.lean` (plus the **Lawson topology**, `LawsonTopology.lean`, and Scott–Hausdorff separation).
- **ω -complete partial orders** — `Order/OmegaCompletePartialOrder.lean` and `Topology/OmegaCompletePartialOrder.lean`: ω CPOs and ω -continuous maps, the directed-complete setting for denotational semantics (referenced across ~ 13 files). Also `Order/CompletePartialOrder.lean`.
- **Fixed-point theory** — `Order/FixedPoints.lean` (Knaster–Tarski least/greatest fixed points, `OrderHom.lfp/gfp`) and `Order/BourbakiWitt.lean`. A `LawfulFix` operator gives the Kleene least fixed point $\bigsqcup_n f^n(\perp)$ of an ω -continuous map (~ 22 files) — Scott’s account of recursion, used to define partial/recursive functions on the `Part` monad.

So the core apparatus — Scott continuity, the Scott topology, cpos, and least-fixed-point recursion — is genuinely formalized.

What it does not have

- **No named “Scott domain.”** There is no `DirectedCompletePartialOrder/DCPO` class (it works through `OmegaCompletePartialOrder`), and no development of bounded-complete algebraic dcpos or Scott information systems.
- **No continuous-lattice theory** under that name (the Lawson topology is present, but not the continuous-lattices development).
- **The big absence: no D_∞ .** The reflexive object $D \cong [D \rightarrow D]$ — the model of the untyped λ -calculus — is not in Mathlib, nor is the category of domains packaged as a cartesian-closed category for denotational semantics. The inverse-limit solution of recursive domain equations has not been formalized.

The upshot

Mathlib has the *ingredients* (Scott continuity, Scott topology, ω CPOs, least fixed points) but not the *signature constructions* of domain theory as a model of computation (D_∞ , the cartesian-closed category of domains, continuous lattices). That gap is an opportunity: formalizing D_∞ in Lean would be a genuinely new — and fitting — contribution.